

Alias Inteligentes (v2.0) para ecosistemas DevSecOps

Bitacora | Vol. 1

mercedev.es — 2026-04-24

Contexto: Al ejecutar el orquestador local (`merci-total`) desde la terminal en un nuevo repositorio derivado (*Merci Boilerplate*), el sistema auditaba por error el proyecto padre original (`mercedev.es`) en lugar del directorio activo. Esto ocurrió porque se habían configurado alias de terminal (`~/.zshrc`) con rutas absolutas. Al ejecutar el script del proyecto padre, Python utilizó `Path(__file__).resolve().parents` para descubrir su propia ubicación física, auditando en consecuencia la carpeta donde residía el archivo y no el directorio actual de la terminal.

Hecho: - Se descartó la práctica de definir alias estáticos (`alias merci-total="python3 /ruta/absoluta/..."`). - Se reemplazaron los alias por una función Bash inteligente sensible al contexto en el perfil de la terminal. - Se diagnosticó y purgó la caché de la sesión de la terminal que retenía en memoria el alias obsoleto, causando falsos positivos durante la prueba.

Detalle técnico: Se inyectó el siguiente bloque de código en el archivo de configuración del usuario (`~/.zshrc` o `~/.bashrc`):

```
# Motor Merci - Ejecutor Inteligente
merci() {
    if [ -f "scripts/merci/merci-$1.py" ]; then
        python3 "scripts/merci/merci-$1.py"
    else
        echo "🛡️ [Merci Error] No estás en la raíz de un proyecto Merci
o el comando '$1' no existe."
    fi
}
```

Al posicionarse en la raíz de cualquier repositorio que contenga la estructura del proyecto, el desarrollador simplemente invoca `merci total` o `merci audit`. La función buscará la ruta relativa e invocará al script correcto.

Nota de depuración (Fantasmas en RAM): Si tras borrar los alias antiguos el sistema sigue ejecutando la ruta anterior (ej. al escribir `merci-total` por costumbre), se debe a que la sesión activa mantiene los alias vivos en la memoria volátil. Para purgar el estado, se debe ejecutar explícitamente `unalias merci-total` o abrir una nueva pestaña de terminal.

Motivo / criterio: Universalidad y portabilidad (DX - Developer Experience). Un alias absoluto rompe el aislamiento del entorno, forzando comandos en repositorios ajenos. Una función "Context-Aware" (Consciente del Contexto) permite trabajar con múltiples copias del Boilerplate en la misma máquina utilizando la misma sintaxis global. Además, comprender el ciclo de vida de los procesos en memoria evita horas de depuración innecesaria.

Fuentes / Bibliografía: - Asistencia mediante IA sobre el comportamiento del método `resolve().parents` en librerías de autodescubrimiento de Python. - Asistencia mediante IA sobre la retención en memoria (RAM) de alias obsoletos en sesiones de Bash/Zsh.

Actualización v2.0: Parámetros Infinitos y Recarga en Caliente

El Desafío (Síntoma): La función original era rígida y no permitía pasar argumentos adicionales (flags como `--verbose`, `-v` o rutas de archivos) a los scripts subyacentes, limitando el uso de herramientas como el orquestador de backups.

La Maniobra (Lógica): Se actualizó la función inyectando la variable de expansión `${@:2}` de Zsh/Bash.

```
merci() {  
    if [ -f "scripts/merci/merci-$1.py" ]; then  
        # Ejecuta el script pasándole todos los argumentos a partir del  
segundo  
        python3 "scripts/merci/merci-$1.py" "${@:2}"  
    else  
        echo "🛡️ [Merci Error] No estás en la raíz o el comando no  
existe."  
    fi  
}
```

El Aprendizaje (Recarga de Terminal): Al modificar el archivo `~/.zshrc` (o `~/.bashrc`), los cambios no se aplican mágicamente a las terminales que ya están abiertas. Para inyectar la nueva configuración en caliente sin tener que reiniciar el sistema o cerrar la ventana, es obligatorio ejecutar el comando de recarga:

```
source ~/.zshrc
```